

**Pengujian Strategi Caching Menggunakan Redis,
CDN, Cache Invalidation, dan Cache Eviction
pada Sistem Rekam Medis Pasien Kritis Rumah
Sakit**

Dosen Pengampuh : RUNAL REZKIAWAN, S.Kom.,M.T



OLEH

Nama	AWALUDDIN
Nim	105841121823
Kelas	6A RPL

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum yang berjudul "**Pengujian Strategi Caching Menggunakan Redis, Memcached, CDN, Cache Invalidation, dan Cache Eviction**" dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep strategi caching serta implementasi Redis dan Memcached sebagai media penyimpanan cache. Selain itu, praktikum ini juga bertujuan untuk mempelajari mekanisme Content Delivery Network (CDN), cache invalidation, dan cache eviction dalam meningkatkan performa sistem serta mempercepat proses akses data.

Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengetahuan mengenai penggunaan Docker sebagai lingkungan pengujian, konfigurasi layanan Redis dan Memcached, implementasi web caching menggunakan Express.js, serta pengujian performa sistem dengan membandingkan akses data melalui database dan cache.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik pada masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai penerapan strategi caching serta pemanfaatan Redis, Memcached, CDN, cache invalidation, dan cache eviction dalam meningkatkan efisiensi serta performa aplikasi.

Makassar, juni 2026

AWALUDDIN

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Rumusan Masalah	4
1.3 Tujuan Pengujian.....	4
1.4 Tujuan Pengujian.....	5
BAB II LANDASAN TEORI	6
2.1 Caching.....	Error! Bookmark not defined.
2.2 Redis.....	Error! Bookmark not defined.
2.3 Memcached	Error! Bookmark not defined.
2.4 Content Delivery Network (CDN)	Error! Bookmark not defined.
2.5 Cache Invalidation.....	8
BAB III METODOLOGI PENELITIAN	11
3.1 Alat dan Bahan	11
3.2 Metode Penelitian.....	Error! Bookmark not defined.
3.3 Skenario Pengujian	12
3.4 Tahapan Pengujian	10
BAB IV PENGUJIAN DAN PEMBAHASAN	15
4.1 Menjalankan Sistem Caching Lab	15
4.2 Pengujian Pengambilan Data Produk (Cache Miss)	15
BAB V PENUTUP	19
5.1 Kesimpulan.....	19
5.2 Saran	19
DAFTAR PUSTAKA.....	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

ada sistem terdistribusi, data sering disimpan pada beberapa server yang saling terhubung untuk meningkatkan keandalan dan ketersediaan layanan. Namun, ketika data berada pada lebih dari satu server, muncul tantangan dalam menjaga konsistensi data dan memastikan layanan tetap tersedia saat terjadi gangguan jaringan atau kegagalan server.

Teorema CAP (Consistency, Availability, dan Partition Tolerance) menjelaskan bahwa sistem terdistribusi tidak dapat secara bersamaan menjamin ketiga aspek tersebut secara penuh. Ketika terjadi gangguan jaringan (partition), pengembang harus memilih antara mempertahankan konsistensi data (Consistency) atau mempertahankan ketersediaan layanan (Availability).

Pada proyek ini dibuat sebuah simulasi Teorema CAP menggunakan Node.js dan Express.js. Sistem terdiri dari dua server yang berfungsi sebagai node penyimpanan data serta satu gateway utama yang mengatur komunikasi antar server. Simulasi ini menunjukkan perbedaan perilaku sistem ketika memilih pendekatan Consistency (CP) dan Availability (AP).

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam proyek ini adalah:

- Bagaimana cara mensimulasikan Teorema CAP menggunakan Node.js?
- Bagaimana implementasi konsep Consistency pada sistem terdistribusi?
- Bagaimana implementasi konsep Availability pada sistem terdistribusi?
- Bagaimana perilaku sistem ketika salah satu node mengalami gangguan?

1.3 Tujuan Pengujian

- Memahami konsep dasar Teorema CAP.
- Mengimplementasikan simulasi Consistency dan Availability.
- Mengetahui perbedaan perilaku sistem CP dan AP.
- Mengamati dampak kegagalan salah satu node terhadap sistem.

1.4 Manfaat Pengujian

- Memberikan pemahaman mengenai sistem terdistribusi.
- Menunjukkan penerapan Teorema CAP secara praktis.
- Menjadi media pembelajaran mengenai Consistency dan Availability.
- Membantu memahami pengelolaan data pada lingkungan multi-server

BAB II

LANDASAN TEORI

2.1 Sistem Terdistribusi

Sistem terdistribusi merupakan sekumpulan komputer atau server yang bekerja secara bersama-sama untuk mencapai tujuan tertentu. Meskipun terdiri dari beberapa komponen yang terpisah secara fisik, sistem ini terlihat sebagai satu kesatuan bagi pengguna.

Pada sistem terdistribusi, setiap node dapat melakukan pemrosesan data dan berkomunikasi melalui jaringan. Sistem ini banyak digunakan pada layanan cloud computing, media sosial, e-commerce, dan aplikasi berskala besar yang membutuhkan ketersediaan tinggi.

Karakteristik utama sistem terdistribusi meliputi:

1. Terdiri dari beberapa node atau server.
2. Berkomunikasi melalui jaringan.
3. Mendukung pembagian beban kerja.
4. Memiliki toleransi terhadap kegagalan.
5. Menyediakan akses data secara bersama-sama.

Dalam proyek ini, sistem terdistribusi disimulasikan menggunakan dua server yang saling berkomunikasi untuk melakukan sinkronisasi data..

2.2 Teorema CAP (CAP Theorem)

CAP Theorem merupakan teori yang diperkenalkan oleh Eric Brewer pada tahun 2000. Teori ini menyatakan bahwa sebuah sistem terdistribusi hanya dapat menjamin dua dari tiga karakteristik berikut secara bersamaan:

1. Consistency (Konsistensi)
2. Availability (Ketersediaan)
3. Partition Tolerance (Toleransi Partisi)

Ketika terjadi gangguan komunikasi antar node (network partition), sistem harus memilih antara mempertahankan konsistensi data atau mempertahankan ketersediaan layanan. CAP Theorem menjadi dasar dalam perancangan berbagai sistem database modern seperti Cassandra, MongoDB, DynamoDB, dan CockroachDB..

2.3 Consistency

Consistency adalah kondisi dimana seluruh node dalam sistem memiliki data yang sama pada waktu yang sama. Ketika sebuah data diperbarui pada satu node, perubahan tersebut harus terlihat pada seluruh node sebelum operasi dianggap berhasil.

Karakteristik Consistency:

1. Data selalu seragam pada seluruh node.
2. Menghindari perbedaan informasi antar server.
3. Menjamin keakuratan data.

Kelebihan:

1. Integritas data lebih terjaga.
2. Mengurangi risiko konflik data.

Kekurangan:

1. Respon sistem dapat menjadi lebih lambat.
2. Jika terjadi gangguan jaringan, beberapa layanan dapat ditolak untuk menjaga konsistensi.

Pada proyek ini, mode Consistency memastikan bahwa data hanya dianggap berhasil ditulis apabila kedua server berhasil menerima pembaruan data..

2.4 Availability

Availability adalah kemampuan sistem untuk tetap memberikan layanan kepada pengguna meskipun terjadi kegagalan pada sebagian node. Dalam pendekatan ini, sistem lebih mengutamakan ketersediaan layanan dibandingkan keseragaman data secara langsung.

Karakteristik Availability:

- 1) Sistem tetap dapat menerima permintaan pengguna.
 - Tidak bergantung pada seluruh node aktif.
 - Respon sistem lebih cepat.
- 2) Kelebihan:
 - Layanan tetap berjalan meskipun terjadi gangguan.
 - Meningkatkan pengalaman pengguna.
- 3) Kekurangan:
 - Data antar node dapat berbeda sementara waktu.
 - Memerlukan mekanisme sinkronisasi tambahan.

Pada proyek ini, mode Availability memungkinkan proses penyimpanan data tetap berhasil walaupun salah satu server tidak dapat diakses.

2.5 Partition Tolerance, Node.js, Express.js, Axios, dan REST API

Partition Tolerance merupakan kemampuan suatu sistem terdistribusi untuk tetap beroperasi meskipun terjadi gangguan komunikasi atau putusnya koneksi antar node dalam jaringan. Pada kondisi ini, sistem harus tetap mampu menjalankan sebagian atau seluruh layanannya walaupun terdapat server yang tidak dapat berkomunikasi dengan server lainnya. Dalam CAP Theorem, Partition Tolerance menjadi aspek yang sangat penting karena kegagalan jaringan merupakan kondisi yang umum terjadi pada sistem terdistribusi.

Untuk mengimplementasikan simulasi CAP Theorem pada penelitian ini digunakan Node.js sebagai platform pengembangan aplikasi. Node.js merupakan runtime JavaScript yang berjalan di sisi server dan memungkinkan pengembangan aplikasi jaringan yang cepat,

ringan, dan efisien. Dengan arsitektur non-blocking I/O, Node.js sangat cocok digunakan untuk membangun aplikasi yang membutuhkan komunikasi antar server secara realtime.

Framework yang digunakan dalam pengembangan sistem adalah Express.js. Express.js merupakan framework web untuk Node.js yang menyediakan berbagai fitur untuk membangun server dan API dengan lebih mudah. Framework ini digunakan untuk mengelola routing, menerima permintaan dari pengguna, serta mengatur proses komunikasi antar node dalam sistem.

Selain itu, digunakan Axios sebagai library HTTP Client yang berfungsi untuk mengirim dan menerima data melalui protokol HTTP. Axios memungkinkan server melakukan pertukaran data dengan server lain secara mudah sehingga proses sinkronisasi data antar node dapat dilakukan dengan baik. Dalam simulasi ini, Axios digunakan untuk mengirim data dari satu server ke server lainnya ketika proses konsistensi dan ketersediaan data dijalankan.

Komunikasi antar server pada sistem dilakukan menggunakan REST API (Representational State Transfer Application Programming Interface). REST API merupakan arsitektur layanan web yang memanfaatkan metode HTTP seperti GET, POST, PUT, dan DELETE untuk melakukan pertukaran data antar aplikasi. Pada proyek simulasi CAP Theorem ini, REST API digunakan sebagai media komunikasi antara Gateway Server, Server Gudang A, dan Server Gudang B dalam proses pengiriman serta sinkronisasi data. Dengan memanfaatkan teknologi-teknologi tersebut, sistem dapat mensimulasikan perilaku Consistency dan Availability pada lingkungan sistem terdistribusi secara efektif.

2.6 Arsitektur Sistem Pada Proyek

Arsitektur sistem pada proyek Simulasi CAP terdiri dari tiga komponen utama:

1. Gateway Server
2. Server Gudang A
3. Server Gudang B

Alur kerja sistem:

1. Pengguna mengirimkan data melalui gateway.

2. Gateway meneruskan data ke server tujuan.
3. Server melakukan penyimpanan data.
4. Sistem menjalankan mekanisme Consistency atau Availability sesuai mode yang dipilih.
5. Hasil sinkronisasi ditampilkan kepada pengguna.

Dengan arsitektur tersebut, simulasi mampu menunjukkan bagaimana keputusan antara Consistency dan Availability mempengaruhi perilaku sistem ketika terjadi kegagalan pada salah satu node

.

BAB III

Analisis Perancangan Sistem

3.1 Analisis Sistem

Sistem yang dibangun merupakan simulasi CAP Theorem pada lingkungan sistem terdistribusi. Simulasi ini bertujuan untuk menunjukkan bagaimana sebuah sistem memilih antara Consistency dan Availability ketika terjadi gangguan komunikasi antar server (Partition).

Sistem terdiri dari tiga komponen utama yaitu Gateway Server, Server Gudang A, dan Server Gudang B. Gateway Server berfungsi sebagai pusat komunikasi yang menerima permintaan dari pengguna kemudian meneruskannya ke server tujuan. Server Gudang A dan Server Gudang B berfungsi sebagai node penyimpanan data yang saling berkomunikasi untuk menjaga konsistensi data.

Pada mode Consistency, sistem memastikan data berhasil tersimpan pada seluruh node sebelum memberikan respon sukses kepada pengguna. Sedangkan pada mode Availability, sistem tetap memberikan respon sukses meskipun salah satu node tidak dapat diakses:

3.2 Arsitektur Sistem

Arsitektur sistem terdiri dari satu gateway dan dua server node yang saling terhubung melalui jaringan.

Alur kerja sistem sebagai berikut:

1. Pengguna mengirimkan permintaan melalui Gateway Server.
2. Gateway Server menerima dan memproses permintaan.
3. Data dikirim ke Server Gudang A dan Server Gudang B.
4. Kedua server menyimpan data yang diterima.
5. Sistem menjalankan mekanisme Consistency atau Availability sesuai mode yang dipilih.
6. Hasil proses dikirim kembali kepada pengguna.

3.3 Struktur Folder Proyek

Struktur folder proyek SIMULASI-CAP terdiri dari beberapa file utama yang mendukung jalannya sistem.

```
SIMULASI-CAP/  
├─ gateway.js  
├─ serverA.js  
├─ serverB.js  
├─ package.json  
├─ package-lock.json  
└─ node_modules/
```

Keterangan:

- gateway.js : Server utama yang menerima request dari pengguna.
- serverA.js : Node penyimpanan data Gudang A.
- serverB.js : Node penyimpanan data Gudang B.
- package.json : Konfigurasi project Node.js.
- package-lock.json : Informasi dependency project.

3.4 Perancangan Proses Sistem

Perancangan proses sistem pada simulasi CAP Theorem menggambarkan alur pengolahan data mulai dari pengguna mengirimkan data hingga data disimpan pada server tujuan. Sistem terdiri dari Gateway Server sebagai pusat pengendali komunikasi, serta Server Gudang A dan Server Gudang B sebagai node penyimpanan data.

Proses dimulai ketika pengguna mengirimkan data ke Gateway Server. Gateway kemudian menerima request dan menentukan mekanisme yang digunakan, yaitu Consistency atau Availability. Selanjutnya Gateway menghubungi server tujuan dan mengirimkan data yang diterima untuk disimpan pada masing-masing node.

Pada mode Consistency, Gateway akan mengirimkan data ke Server Gudang A dan Server Gudang B secara bersamaan. Transaksi dianggap berhasil apabila kedua server berhasil menyimpan data. Jika salah satu server gagal merespons atau gagal menyimpan data, maka transaksi dianggap gagal untuk menjaga konsistensi data pada seluruh node.

Sedangkan pada mode Availability, sistem lebih mengutamakan ketersediaan layanan. Gateway akan mengirimkan data ke server yang masih aktif dan dapat diakses. Meskipun salah satu server mengalami gangguan, sistem tetap memberikan respon berhasil kepada pengguna sehingga layanan tetap tersedia. Sinkronisasi data dapat dilakukan kembali setelah server yang mengalami gangguan kembali aktif.

Secara umum alur proses sistem adalah sebagai berikut:

- Pengguna mengirimkan data ke Gateway Server.
- Gateway menerima dan memproses request.
- Gateway menentukan mekanisme yang digunakan (Consistency atau Availability).
- Gateway mengirimkan data ke Server Gudang A dan/atau Server Gudang B.
- Server menyimpan data yang diterima.
- Hasil proses penyimpanan dikirim kembali ke Gateway.
- Gateway memberikan respon kepada pengguna sesuai hasil proses yang terjadi.

3.5 Perancangan API dan Alur Sistem

Sistem menggunakan REST API sebagai media komunikasi antara Gateway Server, Server Gudang A, dan Server Gudang B. API berfungsi untuk mengirim, menerima, dan menyinkronkan data antar node dalam lingkungan sistem terdistribusi. Melalui API, setiap server dapat saling berkomunikasi menggunakan metode HTTP sehingga proses penyimpanan dan pertukaran data dapat dilakukan secara efisien.

Endpoint yang digunakan dalam sistem meliputi endpoint penyimpanan data dengan metode POST yang berfungsi untuk menambahkan data baru ke server, endpoint pengambilan

data dengan metode GET yang digunakan untuk menampilkan seluruh data yang tersimpan pada server, serta endpoint sinkronisasi dengan metode POST yang berfungsi untuk mengirim data antar node guna menjaga konsistensi sistem.

Secara umum alur kerja sistem dimulai ketika pengguna mengirimkan request ke Gateway Server. Gateway kemudian menerima dan memproses permintaan tersebut sebelum menentukan mode CAP yang digunakan, yaitu Consistency atau Availability. Pada mode Consistency, data akan dikirim dan disimpan pada seluruh node yang terhubung sehingga setiap server memiliki data yang sama. Sebaliknya, pada mode Availability, data akan disimpan pada node yang masih aktif sehingga layanan tetap tersedia meskipun salah satu server mengalami gangguan.

Setelah proses penyimpanan selesai dilakukan, server akan mengirimkan hasil proses kepada Gateway. Selanjutnya Gateway memberikan respons kepada pengguna sebagai tanda bahwa permintaan telah berhasil diproses. Dengan mekanisme tersebut, sistem mampu mensimulasikan penerapan konsep CAP Theorem pada lingkungan sistem terdistribusi.

Adapun alur sistem secara umum dapat dijelaskan sebagai berikut:

1. Pengguna mengirimkan request ke Gateway Server.
2. Gateway menerima dan memproses request.
3. Gateway menentukan mode CAP yang digunakan.
4. Jika mode Consistency dipilih, data dikirim ke seluruh node.
5. Jika mode Availability dipilih, data dikirim ke node yang aktif.
6. Server menyimpan data yang diterima.
7. Hasil proses dikirim kembali ke Gateway.
8. Gateway memberikan respons kepada pengguna.
9. Proses selesai.

BAB IV

PENGUJIAN DAN PEMBAHASAN

4.1 Menjalankan Sistem UGD Caching Lab

Pada tahap ini seluruh layanan dijalankan menggunakan Docker Compose yang terdiri atas aplikasi Node.js, Redis Server, dan Nginx sebagai simulasi Content Delivery Network (CDN). Setelah seluruh container berhasil dijalankan, aplikasi dapat diakses melalui browser pada alamat: <http://localhost:3000>

Halaman utama aplikasi menampilkan sistem Sistem Akselerasi Data Rekam Medis Pasien Kritis (UGD Data Core) yang digunakan untuk melakukan pengujian strategi caching. Pada halaman tersebut tersedia tombol pengambilan data rekam medis pasien, penghapusan cache, pengunduhan log hasil pengujian, metrik transmisi data, serta tampilan data pasien dalam format JSON..

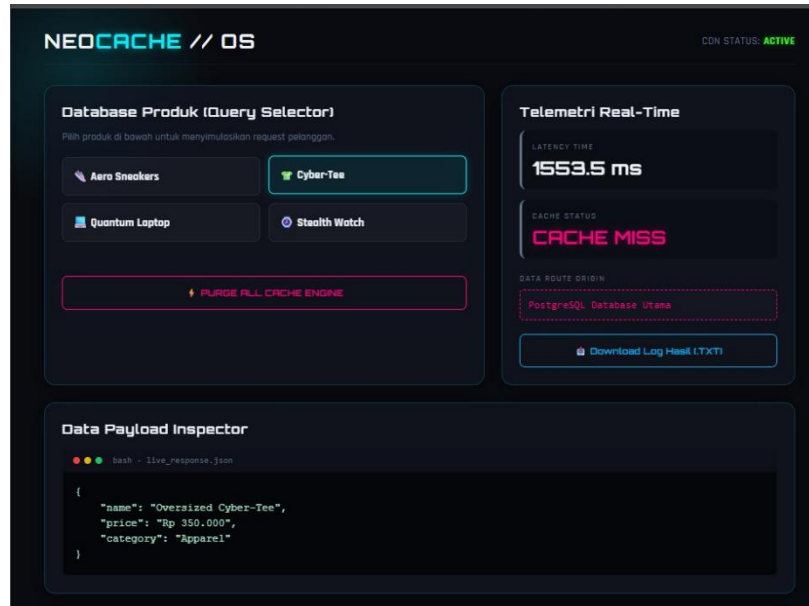
4.2 Pengujian Pengambilan Data Rekam Medis Pasien (Cache Miss)

Hasil pengujian menunjukkan bahwa waktu akses data yang diperoleh adalah sebesar 2328 ms. Pada bagian jalur koneksi ditampilkan status:

DATABASE PUSAT RS (CACHE MISS)

Status tersebut menunjukkan bahwa data belum tersedia pada cache sehingga sistem harus melakukan proses pengambilan data dari sumber utama terlebih dahulu.

Selain itu, sistem berhasil menampilkan data pasien kritis dalam format JSON yang berisi informasi identitas pasien, kondisi kesehatan, golongan darah, riwayat alergi obat, tindakan darurat, dan waktu pembaruan data.



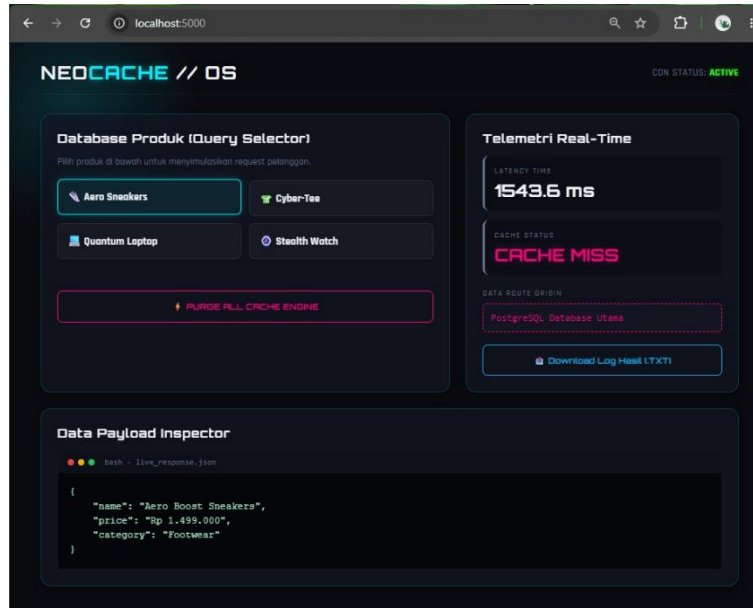
4.3 Pengujian Payload Data Rekam Medis

Setelah proses pengambilan data berhasil dilakukan, sistem menampilkan informasi rekam medis pasien pada bagian Berkas Rekam Medis Terenkripsi (JSON).

Data yang ditampilkan meliputi:

- a. ID Pasien : PASIEN-9921
- b. Nama Pasien : Budi Santoso
- c. Kondisi : Kritis (Gagal Jantung)
- d. Golongan Darah : AB+
- e. Riwayat Alergi Obat
- f. Tindakan Darurat
- g. Waktu Pembaruan Data

Data tersebut menunjukkan bahwa sistem berhasil mengambil dan menampilkan informasi pasien secara lengkap dari sumber data yang tersedia..



4.4 Pengujian Cache Invalidation (Hapus Cache)

Pengujian berikutnya dilakukan dengan menekan tombol Inisialisasi Ulang Sesi (Hapus Cache). Proses ini bertujuan untuk menghapus seluruh data yang tersimpan pada Redis sehingga sistem kembali berada pada kondisi awal. Setelah tombol ditekan, sistem menampilkan pesan: Sesi Cache Jaringan Medis berhasil dibersihkan!

Pesan tersebut menunjukkan bahwa proses Cache Invalidation berhasil dijalankan dan seluruh data cache telah dihapus dari memori Redis.

4.5 Hasil Pengujian Setelah Cache Dihapus

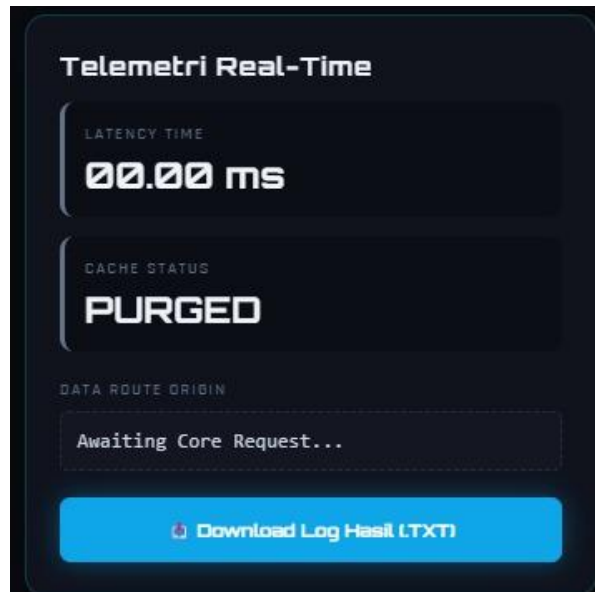
Setelah proses penghapusan cache selesai dilakukan, sistem menampilkan kondisi bahwa sesi pemantauan telah dibersihkan dan cache berada dalam keadaan kosong. Pada bagian metrik transmisi data tidak lagi ditampilkan nilai kecepatan akses maupun jalur koneksi karena tidak terdapat data yang sedang tersimpan di dalam cache. Informasi yang muncul pada layar adalah: Sesi pemantauan dibersihkan. Cache kosong.

Kondisi tersebut menunjukkan bahwa proses penghapusan cache berhasil dilakukan dan sistem siap melakukan pengambilan data ulang dari database pada permintaan berikutnya.

4.6 Pengujian Fitur Unduh Log Hasil

Aplikasi menyediakan fitur **Unduh Bukti Log Uji (.TXT)** yang berfungsi untuk

menyimpan hasil pengujian ke dalam berkas teks. Fitur ini dapat digunakan sebagai dokumentasi aktivitas sistem selama proses pengujian berlangsung. Melalui fitur ini pengguna dapat menyimpan informasi mengenai waktu akses data, status cache, serta jalur koneksi yang digunakan selama proses pengambilan data rekam medis pasien.



4.7 Analisis Hasil

Berdasarkan hasil pengujian yang telah dilakukan, sistem berhasil mengimplementasikan mekanisme caching menggunakan Redis pada aplikasi rekam medis pasien kritis. Ketika data pertama kali diakses, sistem menampilkan status **Cache Miss** karena data belum tersedia pada Redis. Pada kondisi tersebut sistem mengambil data langsung dari database pusat rumah sakit dengan waktu akses sebesar **2328 ms**.

Sistem juga berhasil menampilkan data pasien dalam format JSON yang berisi informasi medis penting sehingga dapat digunakan sebagai simulasi akses data pada lingkungan rumah sakit

Proses **Cache Invalidation** berjalan dengan baik. Setelah pengguna menekan tombol penghapusan cache, seluruh data yang tersimpan pada Redis berhasil dihapus dan sistem kembali berada pada kondisi awal. Hal ini ditunjukkan oleh munculnya pesan bahwa sesi cache berhasil dibersihkan serta tampilan cache kosong pada antarmuka aplikasi.

Secara keseluruhan, implementasi Redis pada sistem UGD Data Core berhasil menunjukkan mekanisme kerja caching yang meliputi proses pengambilan data dari database, penyimpanan sementara pada cache, serta penghapusan cache untuk menjaga konsistensi data. Penggunaan caching dapat membantu meningkatkan efisiensi akses data dan mengurangi beban sistem pada aplikasi pelayanan Kesehatan.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil pengujian yang telah dilakukan pada Sistem Akselerasi Data Rekam Medis Pasien Kritis (UGD Data Core), dapat disimpulkan bahwa implementasi strategi caching menggunakan Redis berhasil diterapkan dengan baik pada aplikasi. Sistem mampu melakukan pengambilan data rekam medis pasien kritis dari database pusat rumah sakit dan menampilkan informasi pasien secara real-time dalam format JSON.

Hasil pengujian menunjukkan bahwa ketika data belum tersedia pada Redis, sistem akan mengambil data langsung dari database utama dengan status Cache Miss. Data yang berhasil diperoleh kemudian dapat disimpan sementara pada cache sehingga proses akses data berikutnya dapat dilakukan dengan lebih cepat. Selain itu, mekanisme Cache Invalidation juga berhasil dijalankan melalui fitur penghapusan cache sehingga data yang tersimpan dapat dibersihkan ketika diperlukan.

Penerapan Redis sebagai media caching memberikan manfaat dalam mengurangi beban akses terhadap database utama dan meningkatkan efisiensi sistem. Simulasi CDN menggunakan Nginx juga membantu mendukung distribusi layanan secara lebih optimal. Secara keseluruhan, praktikum ini berhasil menunjukkan bagaimana strategi caching dapat digunakan untuk meningkatkan performa aplikasi, khususnya pada sistem informasi kesehatan yang membutuhkan akses data secara cepat dan stabil.

5.2 Saran

Berdasarkan hasil pengujian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan sistem pada masa mendatang, yaitu:

1. Menambahkan fitur Cache Hit yang lebih informatif sehingga pengguna dapat membandingkan secara langsung perbedaan waktu akses antara data yang berasal dari database dan data yang berasal dari Redis.
2. Mengembangkan sistem dengan menggunakan database nyata seperti PostgreSQL atau MySQL agar simulasi lebih mendekati kondisi implementasi pada lingkungan rumah

sakit sebenarnya.

3. Menambahkan dashboard monitoring Redis untuk memantau penggunaan memori, jumlah request, dan status cache secara real-time.
4. Mengimplementasikan CDN publik seperti Cloudflare agar proses distribusi aset statis dapat dilakukan secara lebih optimal pada jaringan yang lebih luas.
5. Melakukan pengujian performa menggunakan tools seperti Apache JMeter atau Locust untuk mengetahui kemampuan sistem dalam menangani jumlah pengguna yang lebih banyak.
6. Menambahkan variasi data pasien yang lebih besar sehingga pengujian dapat memberikan hasil yang lebih representatif terhadap kondisi operasional sesungguhnya.

Dengan pengembangan tersebut, sistem diharapkan mampu memberikan performa yang lebih baik, meningkatkan efisiensi pengelolaan data, serta mendukung pelayanan kesehatan yang lebih cepat dan akurat.

DAFTAR PUSTAKA

- Docker Inc. (2026). *Docker Documentation*. <https://docs.docker.com/>
- Redis Ltd. (2026). *Redis Documentation*. <https://redis.io/docs/>
- NGINX. (2026). *NGINX Documentation*. <https://nginx.org/en/docs/>
- Mozilla Developer Network. (2026). *HTTP Caching*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching>
- Cloudflare. (2026). *What is Caching?* <https://www.cloudflare.com/learning/cdn/what-is-caching/>
- Cloudflare. (2026). *What is a CDN?* <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th Edition). McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.
- Fielding, R., & Reschke, J. (2022). *Hypertext Transfer Protocol (HTTP/1.1): Caching*. Internet Engineering Task Force (IETF).
- Tanenbaum, A. S., & Wetherall, D. J. (2018). *Computer Networks* (5th Edition). Pearson Education.